

This is CS50

Week 3

Kelly Ding (she/her)

Preceptor

kelly@cs50.harvard.edu

- What are **structs**?
- How does **recursion** work?
- How can we **search** and **sort** efficiently?
- What is **Big O notation**?

Part 1

Structs

How can we group different
types of related data into a
single unit?

```
typedef struct
{
    string name;
    int votes;
}
candidate;
```

```
typedef struct
{
    string name;
    int votes;
}
candidate;
```

```
typedef struct
{
    string name;
    int votes;
}
candidate;
```



```
typedef struct
{
    string name;
    int votes;
}
candidate;
```

candidate president;

candidate president;

type

candidate president;

name

```
candidate president;  
president.name = "Alice";  
president.votes = 10;
```

Questions?



Struct Exercise

Create a struct to represent a candidate in an election that minimally includes:

- The candidate's name (as a string)
- The candidate's number of votes (as an integer)

Add attributes to a candidate and print those out to the user.

```
$ ./struct  
Alice has a 5 votes.
```

Structs and Functions Exercise

Create your own `get_candidate` function that prompts the user to input attributes for a candidate. You may rely on `get_string`, `get_float`, etc., and your function should return a candidate.

```
$ ./struct  
Name: Alice  
Votes: 5  
Alice has a 5 votes.
```


What if we had multiple candidates?

name			
votes			

```
candidate candidates[3];
```

name			
votes			

`candidates[0];`

name	Alice		
votes			

```
candidates[0].name = "Alice";
```

name	Alice		
votes	2		

```
candidates[0].votes = 2;
```

Questions?



Arrays of Structs Exercise

Use your `get_candidate` function to create an array of three candidates, each of which should have attributes input by the user.

```
$ ./struct
Name: Alice
Votes: 5
Name: Bob
Votes: 10
Name: Charlie
Votes: 8
Candidate 1 is named Alice and has 5 votes.
Candidate 2 is named Bob and has 10 votes.
Candidate 3 is named Charlie and has 8 votes.
```

Part 2

Recursion

Recursion

Solving a problem by first solving a smaller version of the same problem.

Every recursive function should have:

- **Base case:** when to stop running the function.
- **Recursive case:** a call to the function to solve a smaller version of the problem.

Factorial Example

Write a recursive function, `factorial`, that takes an integer `n`, and prints its factorial.

The factorial of 5, for example, is $5 * 4 * 3 * 2 * 1 = 120$.

Factorial

$$3! = 3 * 2 * 1 * 1$$

$$2! = 2 * 1 * 1$$

$$1! = 1 * 1$$

$$0! = 1$$

Factorial

$$3! = 3 * 2!$$

$$2! = 2 * 1!$$

$$1! = 1 * 0!$$

$$0! = 1$$

Factorial

$$f(3)$$

$$3 * f(2)$$

$$2 * f(1)$$

$$1 * f(0)$$

$$1$$

Factorial

$f(3)$

$3 * f(2)$

$2 * f(1)$

$1 * f(0)$

1



1

Factorial

$$f(3)$$

$$3 * f(2)$$

$$2 * f(1)$$

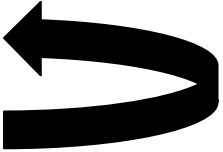
$$1 * 1$$



1

Factorial

$f(3)$

$3 * f(2)$
 $2 * 1$  2

Factorial

$$\begin{array}{c} f(3) \\ 3 * 2 \end{array} \quad \curvearrowleft \quad 6$$

Factorial

6

Factorial

What's the base case?

What's the recursive case?

Questions?



Fibonacci Exercise

Write a recursive function, `fib`, that takes an integer `n`, and prints the `n`th Fibonacci number.

The Fibonacci sequence is a sequence in which each element is the sum of the two elements that precede it:

(0, 1, 1, 2, 3, 5, 8, 13, 21,...)

Part 3

Searching & Sorting

How would you search for 6?





1	8	?	?	?	?	?	?
---	---	---	---	---	---	---	---

1	8	3	?	?	?	?	?
---	---	---	---	---	---	---	---

1	8	3	6	?	?	?	?
---	---	---	---	---	---	---	---

Runtime analysis

How many checks are needed to search for 6 in this list?



What about this list?

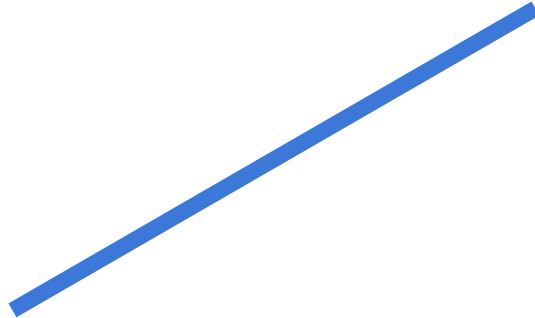


$O(n)$ — "upper bound" definition

I need to do as many as **n** steps for an input of size **n** .

$O(n)$ — "scaling" definition

For every new item that gets added to my algorithm's input, the algorithm needs to do a fixed number of new steps. We say "our runtime scales **linearly** with the size of our input".



Questions?



Sorting

Selection Sort

min



min

5

5

3

4

8

2

1

7

6



min

3

5

3

4

8

2

1

7

6



min

2

5

3

4

8

2

1

7

6



min

1

5

3

4

8

2

1

7

6



min



min

3

1

3

4

8

2

5

7

6



min

2

1

3

4

8

2

5

7

6



min



min

4

1

2

4

8

3

5

7

6



min

3

1

2

4

8

3

5

7

6



min



min

8

1

2

3

8

4

5

7

6



min

4

1

2

3

8

4

5

7

6



min



min

8

1

2

3

4

8

5

7

6



min

5

1

2

3

4

8

5

7

6



min



min

8

1

2

3

4

5

8

7

6



min

7

1

2

3

4

5

8

7

6



min

6

1

2

3

4

5

8

7

6



min



min

7

1

2

3

4

5

6

7

8



min

8

1

2

3

4

5

6

7

8

1 2 3 4 5 6 7 8

Repeat for every element in our list, except last:

 Assume the element at this position is the smallest

 For every element to the right:

 If less than our current smallest:

 Update the current smallest

 Swap the current smallest with the current element

```
For i from 0 to n - 2
  Set min = i
  For j from i + 1 to n - 1
    If array[j] < array[min]
      Update min = j
  Swap array[i] with array[min]
```

$O(n^2)$ — "upper bound" definition

I need to do as many as n^2 steps if my input size is n .

$\Omega(n^2)$ — "lower bound" definition

In the best case, I need to do approximately n^2 steps if my input size is n .

Bubble Sort



Are these out of order?







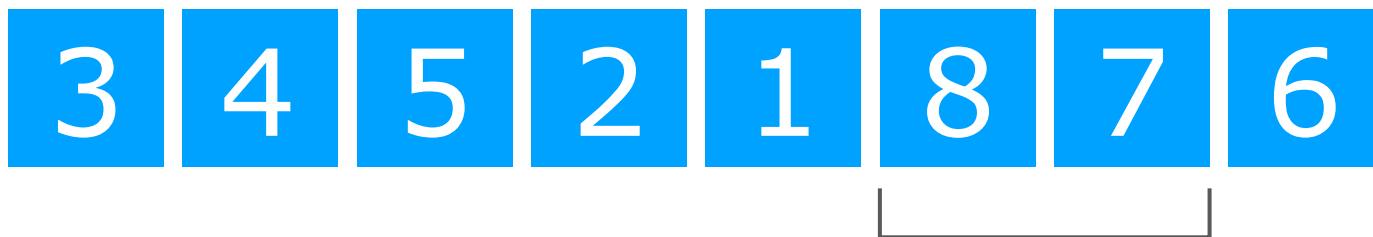




















First pass complete!

3	4	5	2	1	7	6	8
---	---	---	---	---	---	---	---

3	4	2	5	1	7	6	8
---	---	---	---	---	---	---	---

3	4	2	1	5	7	6	8
---	---	---	---	---	---	---	---

3	4	2	1	5	6	7	8
---	---	---	---	---	---	---	---

3	4	2	1	5	6	7	8
---	---	---	---	---	---	---	---

3	2	4	1	5	6	7	8
---	---	---	---	---	---	---	---







2	1	3	4	5	6	7	8
---	---	---	---	---	---	---	---







1 2 3 4 5 6 7 8

Repeat for every element in our list, except last:

Look at each element from first to second-to-last:

If current and next elements out of order:

Swap them

If no two elements were swapped:

Quit

Repeat $n - 1$ times

Set swaps to none

For j from 0 to $n - 2$

 If j 'th and $j + 1$ 'th elements out of order

 Swap them

If swaps = none

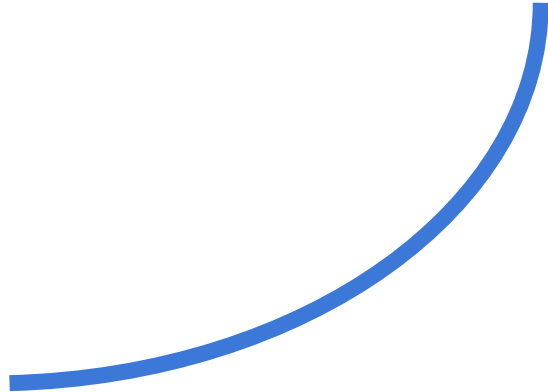
 Quit

$O(n^2)$ — "upper bound" definition

I need to do as many as n^2 steps if my input size is n .

$O(n^2)$ – "scaling" definition

For every new item that gets added to my input, I need to do approximately **n** new steps.



$\Omega(n)$ — "lower bound" definition

In the best case, I need to do approximately **n** steps if my input size is **n** .

Merge Sort

5	3	4	8	2	1	7	6
---	---	---	---	---	---	---	---

5

3

4

8

2

1

7

6



5 3 4 8

2 1 7 6



5 3 4 8

2 1 7 6



























































3 4 5 8

2 1 7 6

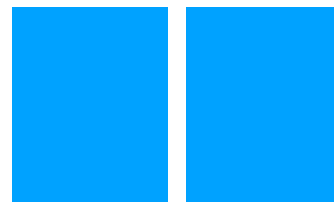


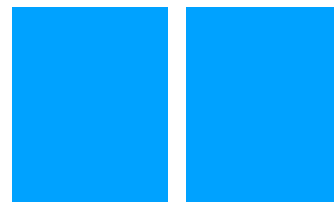
3 4 5 8

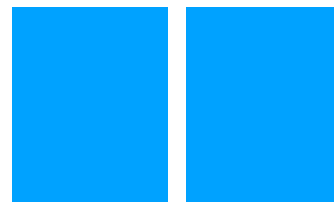
2 1 7 6

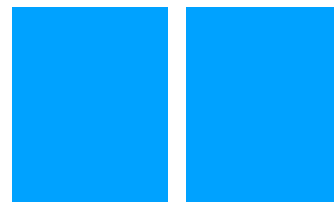


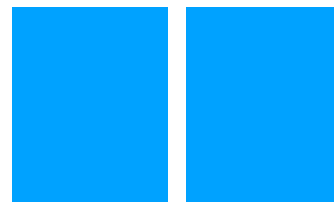




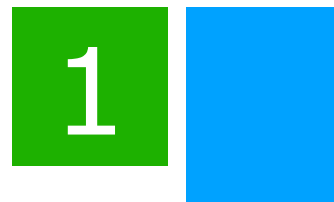




















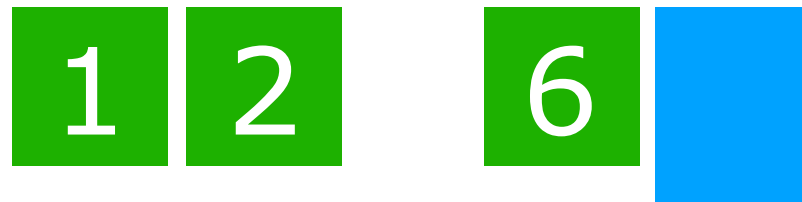






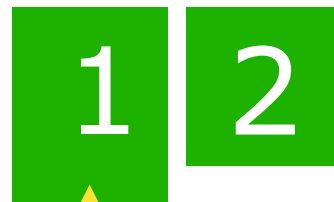
























3 4 5 8

1 2 6 7





3 4 5 8

1 2 6 7

















1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---



1

2

3

4

5

6

7

8

1 2 3 4 5 6 7 8

Split the list into two halves

Sort the left half using merge sort

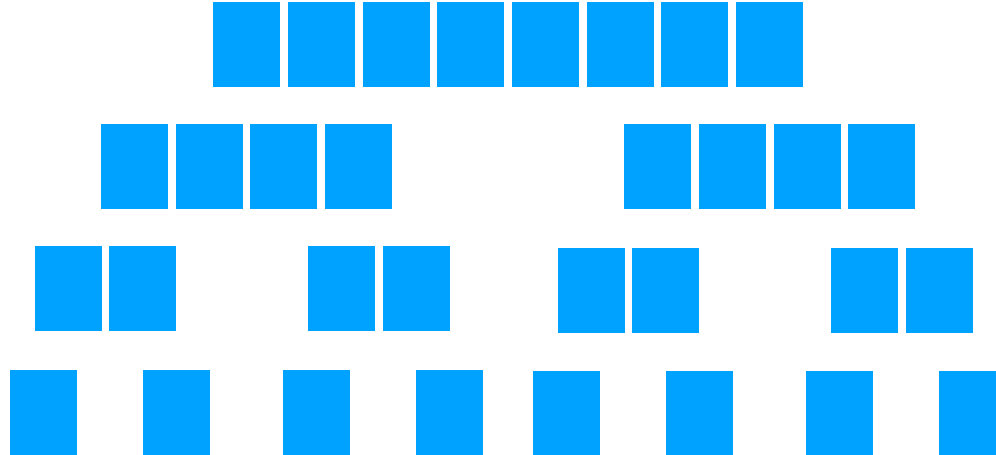
Sort the right half using merge sort

Merge the two halves into a single sorted list:

Take the smaller first element from either half

$O(n \log n)$ — "upper bound" definition

I need to do as many as **$n \log n$** steps to find my solution.



Runtime Analysis

Algorithm	O	Ω
Merge Sort	$O(n \log n)$	$\Omega(n \log n)$
Selection Sort	$O(n^2)$	$\Omega(n^2)$
Bubble Sort	$O(n^2)$	$\Omega(n)$

This is CS50

Week 3